

NAME(S):

Objectives

- To think about writing algorithms
- To practice with modular programming by writing some basic Python functions
- To practice with third-party functions that are pre-built into structures like Strings
- To practice with some useful bioinformatics databases

Reminders for Pair Programming

Pair programming guidelines:

- Roles:
 - driver: has the keyboard and types
 - navigator: guides the driver
- Instructions:
 - You should be talking to each other constantly!
 - You should be able to swap roles at any moment.
 - Swap roles every 5 to 10 minutes.

Part 1: Developing an Algorithm

Given a DNA nucleotide sequence of any length, write an algorithm that would find whether or not it contains a stop codon anywhere in the sequence. Print “YES” if so, and “NO” if not. Note: there are **three** possible codons that can terminate a coding region. They are listed in Figure 1.9 of the Dummies book. Your algorithm should find the first occurrence of any of them. Please write your answer in **pseudocode** using the “limited vocabulary” handout we used in class. You may use additional storage locations (i.e. variables) if you need them. Please type your answer as a section of comments in the Python code you will be writing in Part 2.

- Related context question: Other than stop codons, are there other sets of codons whose sequences result in the same result during protein synthesis?

.....
.....
.....
.....
.....
.....

Part 2: Getting to know Python

Write a Python function named `getCodon` that takes two parameters. The first, named `sequence`, is a nucleotide sequence (a string). The second, named `codonNumber`, is an integer. This function will print out the codon of the nucleotide sequence that corresponds to the number in `codonNumber`. For example, let's say the sequence was `ATCGCTACA`. The zero-th codon is `ATC`. (Remember, computer scientists always start counting from zero.) The first codon is `GCT` and the second is `ACA`. Thus, if `codonNumber` is 1, your function should print `GCT` (codon #1). Your function should work for any nucleotide sequence of any length. You may assume that `codonNumber` is a valid number (it won't be negative or too big).

Hint: to help develop the logic, right down a few examples and compare the nucleotide positions to the codon positions. See if you notice a pattern. Test `getCodon` by writing a `main` function that calls `getCodon` several times with various different sequences of differing lengths. For example: `getCodon("TTGTAA", 1)` should print `TAA`. I'll be looking for your tests.

- Related context question: Explain why it is possible that a deletion of 2 nucleotides is likely to result in a greater change in the eventual protein product of gene X, as compared to a larger deletion of 24 nucleotides from the same region of gene X.

.....
.....
.....

Part 3: Using Tools and Algorithms Together

In this last part, you'll use some common bioinformatics web sites to get real data to test for a Python function you write yourself. First, write a function named `getLastCodon`. It will take one parameter: a nucleotide sequence. This function will print the last codon in the sequence, regardless of sequence length. You may assume the sequence length is divisible by 3 so that the last three characters in the string make up the last codon. You should test it on your own made-up sequences by calling it from your main function. Make sure to leave your tests in your code so I can see them. After that, get some real test data by doing the following:

1. There is a gene that has been linked to prostate and ovarian cancer called `BRCA2`. Read an article about it at [BBC News](#).
2. Use the NCBI web page (<https://www.ncbi.nlm.nih.gov/>) to find a `BRCA2`-related article entitled 'Scientists find gene linking inherited breast and ovarian cancer to non-inherited cases' by Susan Mayor. In this article, they describe the findings of a gene that binds to `BRCA2`, potentially triggering the onset of cancer.
3. Once you know the name of this gene (named for a nurse who works in the field), use the Swiss-Prot protein database (<https://www.uniprot.org/>) to find the associated protein created from this gene in humans. Use the example described in Chapter 2 of the Dummies book as a guide. Make sure to find the **canonical** form of this protein, which is the most widely expressed variant.

- **Hint:** the canonical form has 1322 amino acids.

4. Once you find the amino acid sequence of the protein, you can find the corresponding nucleotide sequence by going to the Consensus Conserved Domain Server (CCDS). You can get to the correct portion of the web page by clicking on “Cross-references” in the left-hand navigation bar.
 - **PITFALL ALERT:** Make sure you get the nucleotide sequence corresponding to the canonical protein sequence. Use the canonical form’s identifier to help. When you click the correct option, you’ll be taken to a page with the nucleotide sequence.
5. This nucleotide sequence in the CCDS is the sequence with which you should test the `getLastCodon` function. Copy and paste it into your program as test input. If a string extends onto multiple lines, you can get Python to treat the next line as a continuation of the current line by putting a backslash (`\`) at the end of each line.
 - **BUSYWORK BUSTER:** If you put this long string into a function, you just need to indent the first line.

Be sure to keep this sequence in your program when you turn it in. I’ll be looking for it.

Share your code between partners so you can each finish it outside of class if you need to. Individually submit to gradescope.

- Related context question: Describe as many different types of mutations as you can and compare the relative severity of their effects on protein synthesis.

.....

.....

.....

.....

.....

.....

- Related context question: Explain how you can obtain wild-type progeny from a cross involving two mutant individuals with identical “abnormal” phenotypes. You can assume that both the parents and the progeny are diploid. What are the genotypes of the two mutant parents and their progeny. Hint: think about possible modes of inheritance that would be consistent with this data.

.....

.....

.....

.....

.....

.....

.....